

Do Internal Failures Predict External Ones? Causal Hop Localization for Faithful, Efficient Retrieval-Augmented Multi-Hop QA

Navadeep Kiran Havish Balaga Anish Racherla Yogansh
International Institute of Information Technology, Hyderabad

Abstract

Large language models (LLMs) performing multi-hop reasoning in retrieval-augmented generation (RAG) settings often fail silently: an early intermediate fact is guessed incorrectly, and the error propagates through the rest of the reasoning chain while the model continues to generate fluent, confident-sounding text. Existing repair frameworks, such as Doctor-RAG, diagnose these failures only after the fact, by reading the model’s completed output text and inferring where reasoning likely broke down. This is both computationally wasteful (a full, faulty trajectory must be generated before it can be diagnosed) and epistemically indirect (the diagnosis is based on an external textual signal rather than directly probing the model’s internal representations). We propose to test whether a lightweight probe trained on the model’s internal hidden states can detect a failing reasoning hop at the boundary of the current hop, before subsequent reasoning steps are generated, and whether this internal signal can be causally tested, rather than merely correlated with failure, using activation patching. We further propose to use this probe as a real-time trigger for a “Naive Restart” retrieval-augmented repair mechanism, and to compare it against a re-implemented text-based diagnosis baseline in the style of Doctor-RAG.

1 Motivation

Agentic Retrieval-Augmented Generation (RAG) enables LLMs to answer complex, multi-hop questions by decomposing them into intermediate reasoning steps (“hops”), retrieving evidence, and reasoning sequentially (Lewis et al., 2020). Errors made early in this chain compound: if the model misidentifies an intermediate bridging entity, subsequent hops depending on it are likely wrong too, even though the generated text may remain fluent and confident.

Recent diagnostic frameworks such as Doctor-RAG (Jiao et al., 2026) address these failures by

analyzing the model’s complete generated Chain-of-Thought (CoT) trajectory after generation, using a secondary model to identify whether a failure occurred and localize its earliest point. This is fundamentally reactive: the full trajectory must be generated before diagnosis and repair can begin, and because the diagnosis is text-based, it may not faithfully reflect when the internal reasoning process first deviated.

Mechanistic interpretability research offers an alternative: Probing-RAG (Baek et al., 2025) shows hidden-state representations can predict whether external retrieval is necessary before generation completes. However, this has only considered coarse, query-level decisions, not fine-grained localization of failures within individual hops.

This reveals a gap: it is unclear whether hidden states can identify *which* hop fails early enough to intervene before further erroneous reasoning is generated – and even if a probe predicts failure, predictive performance alone does not establish the representation is causally involved, rather than merely correlated with the outcome.

This project investigates whether a lightweight probe on an LLM’s hidden states can detect and localize failing hops at the hop boundary, before subsequent steps are generated. We use activation patching (Meng et al., 2022; Vig et al., 2020) to test whether the identified representation is causally related to the outcome, and whether this signal can serve as a real-time trigger for retrieval-based repair, compared directly against a text-based approach under matched conditions.

2 Related Work

Prior work on detecting reasoning failures in multi-step LLM systems differs along two axes: intervention granularity (trajectory-level vs. step-level) and signal source (generated text vs. internal computation).

Trajectory-level, text-based diagnosis. Doctor-RAG (Jiao et al., 2026) reads a complete generated trajectory and localizes the earliest failure post hoc via a secondary model – avoiding per-step intervention, but requiring a full trajectory before any repair can begin.

Step-level, text-based verification. Process reward models such as Lightman et al. (2024) score each reasoning step from its generated text via a separately trained verifier. This is more granular than trajectory-level methods, but the signal remains textual: these methods cannot detect a failure the model’s own text never reveals, and online use still requires access to the agent’s step-by-step execution loop.

Query-level, internal-state signals. Probing-RAG (Baek et al., 2025) shows hidden states can predict whether retrieval is needed before generation completes, but this has not been extended to hop-level localization, nor causally validated.

Our work occupies the fourth combination: step-level granularity with an internal-state signal, verified causally rather than assumed from correlation. Unlike text-based step-level methods, our approach requires white-box access to hidden states and cannot apply to closed-source systems.

3 Research Questions

RQ1 (Detection & Localization): Can a lightweight probe on an LLM’s hidden states detect and localize a failing reasoning hop at its boundary, before subsequent steps are generated?

RQ2 (Causality): Is this identified representation causally responsible for the downstream failure (via activation patching), or merely correlated?

RQ3 (Comparative Efficacy): How does this internal trigger compare to a text-based baseline (Doctor-RAG) in failure-detection accuracy, localization accuracy, latency/cost, and post-repair accuracy?

RQ4 (Robustness): At which layer(s) does failure first become linearly decodable, and does the internal signal remain useful (e.g., for faster halting) even if precise localization is imperfect?

4 Contributions

This project makes the following contributions:

- A lightweight hidden-state probing approach for **hop-level failure localization**, identifying

the earliest reasoning hop where the model’s internal computation deviates from a correct path, before subsequent hops are generated.

- Evaluation of this signal via **activation patching**, testing whether restoring the corresponding hidden state can causally change the model’s downstream reasoning outcome, going beyond correlation.
- A real-time **retrieval-and-restart intervention** that interrupts generation upon detecting a failing hop, retrieves relevant evidence, and restarts before further erroneous reasoning is produced.
- A controlled comparison between **internal-state-based** and text-based failure detection under matched conditions, evaluating detection accuracy, localization, latency, token cost, and end-to-end answer accuracy.

5 Proposed Methodology

Our methodology proceeds in five phases: data construction and labeling, internal-state probing (diagnosis), causal verification (activation patching), runtime intervention (the “Naive Restart” RAG system), and evaluation against a re-implemented Doctor-RAG-style text-based failure-localization baseline. We use a 3B-parameter instruction-tuned base model (Llama-3.2-3B or Qwen2.5-3B) (Dubey et al., 2024; Yang et al., 2024) at float16 precision.

5.1 Data Pipeline and Hop-Level Labeling

We will construct a labeled dataset of multi-hop question-answer pairs, primarily from **2WikiMultihopQA** (Ho et al., 2020), chosen because its explicit, structured ground-truth reasoning graphs make automatic hop-level labeling tractable. **HotpotQA** (Yang et al., 2018) will serve as an out-of-domain evaluation set for final-answer failure prediction and end-to-end accuracy, since it lacks the same structured reasoning graph.

Hop-level labeling. After the model generates each hop, we extract the bridging entity it mentions and compare it against the corresponding ground-truth node in 2WikiMultihopQA’s reasoning graph. A hop is labeled successful when this entity reliably matches the gold entity. Since exact-string matching is too brittle against valid paraphrases, we use normalized matching, a Wikidata alias table, and a Sentence-BERT semantic-similarity fallback (Reimers and Gurevych, 2019).

Class balancing via counterfactual trick questions. Since a competent 3B-parameter model resolves most hops correctly, natural failures will be sparse. We will construct minimal counterfactual edits of real questions (e.g., substituting one entity for another of matched token length) designed to induce a specific hop failure, validating each by confirming the intended failure against ground truth. These give balanced negative examples for probe training, and the same pairs are reused as the clean/corrupted pairs for causal verification in Phase 2, avoiding duplicated data construction.

Prompt formatting. Base models are prompted to wrap intermediate steps in XML-style tags (e.g., `<hop1> [text] </hop1>`). This ensures deterministic bridging entity extraction and precise token boundaries for pooling.

5.2 Phase 1: Diagnosis via Internal Probing

Token-boundary detection. We detect hop boundaries using a character-level buffer, since the tokenizer may split the hop delimiter across multiple tokens.

Multi-layer extraction. We extract hidden states from *multiple* layers in the same forward pass using `nnsight` hooks, and compare which layer(s) carry the strongest signal. This supports our fall-back analysis (Section 5.5) with modest added storage and probe-evaluation overhead but no additional forward passes.

Pooling. For classifier stability, hidden states spanning each hop are mean-pooled:

$$x_{\text{probe}} = \frac{1}{N} \sum_{i=t_{\text{start}}}^{t_{\text{end}}} h_i^{(l)}$$

where $h_i^{(l)}$ is the hidden state of the i -th token at layer l . We note that pooling is appropriate here specifically because it is used for classifier training, not for the causal intervention in Phase 2, where a single-token representation is required instead (Section 5.3).

Probe types. We will train and compare two probe families on the pooled vectors: (1) a linear logistic regression probe, and (2) a small multi-layer perceptron (MLP) probe.

5.3 Phase 2: Causal Verification via Activation Patching

A classifier that correlates with failure is not sufficient; we must show the probed representation is *causally* responsible for the reasoning outcome. We adopt the activation patching / causal tracing methodology from prior interpretability work (Meng et al., 2022; Vig et al., 2020).

1. **Minimal pairs.** We construct clean and corrupted prompt pairs. To guarantee exact positional alignment despite BPE tokenizer sensitivity, we programmatically enforce strict token-by-token prefix matching up to the target entity being swapped, rather than just matching total length.
2. **Clean extraction.** For each clean prompt, we save the hidden state at the target layer and at a specific token position, chosen as the final token of the bridging-entity mention. This provides a consistent and semantically motivated intervention point for the causal analysis. This is distinct from the mean-pooled representation used for classifier training, since causal patching requires a precise token-level intervention rather than an aggregate representation.
3. **Patch.** We run the corrupted prompt and, using `nnsight`, overwrite the hidden state at the same token position and layer with the saved clean vector.
4. **Verification.** If the model’s final answer flips from incorrect to correct as a result of this single-position patch, we take this as evidence that the patched representation is causally responsible for the reasoning outcome at that hop.

We will utilize a meticulously validated set of minimal pairs for this phase, prioritizing the rigorous manual verification required for activation patching to establish a robust proof-of-concept for our causal claims.

5.4 Phase 3: Runtime Intervention (Naive Restart)

We integrate the trained probe into a live generation loop as follows.

1. **Custom generation loop.** We implement token-by-token generation using `model.forward()` directly, since we need to pause mid-sequence.
2. **Live probing.** At each hop boundary, we

mean-pool the hop’s hidden states and pass them through the trained probe to obtain $P(\text{failure})$.

3. **Interrupt.** If $P(\text{failure})$ exceeds a calibrated threshold τ , generation halts. Rather than an arbitrary static value, τ is empirically calibrated on our validation split to maximize $F_{0.5}$, penalizing false-positive interruptions to save latency.
4. **Retrieval.** Retrieval is driven by the hop’s textual context, not the hidden state itself, which is used only for failure detection. The partially generated hop text is embedded (all-MiniLM-L6-v2, Reimers and Gurevych, 2019) and used to query a FAISS index (Johnson et al., 2019) over a Wikipedia passage subset, retrieving the top- k ($k=3$) chunks.
5. **Naive Restart.** Rather than splicing retrieved evidence into the existing KV cache mid-generation – which risks silent attention-coherence degradation – we flush the cache entirely, prepend the retrieved evidence to the original prompt, and restart the full forward pass. This trades token efficiency for architectural reliability.

5.5 Fallback Analysis Plan

If hop-level localization does not succeed cleanly, we will still analyze, using the same extracted data: which layer first makes failure linearly decodable, whether hidden states predict eventual final-answer failure even without precise hop localization, and whether our method detects failure earlier than the text-based baseline even when localization is imperfect. A result where our method is faster but not strictly more accurate is a valid, reportable outcome.

5.6 Phase 4: Baseline – Approximated Doctor-RAG

To provide a controlled comparison, we implement a text-based failure-detection baseline in the spirit of Doctor-RAG (Jiao et al., 2026) – a controlled approximation of the diagnose-and-repair paradigm.

1. Generate the complete CoT trajectory using the same base model and prompt template as our system, with no interruption.
2. Pass the complete generated trajectory to a second, few-shot-prompted call of the same base model, asking it to determine whether a failure occurred and, if so, to localize the

earliest failed hop.

3. If a failure is flagged, retrieve using the same FAISS index and embedder built for Phase 3, and restart generation with the retrieved evidence injected.

The primary experimental difference is the failure-detection trigger: internal hidden states versus generated text.

5.7 Phase 5: Experimental Setup and Evaluation

Setup & Datasets. We use Llama-3.2-3B-Instruct or Qwen2.5-3B-Instruct (float16) as our base models. Retrieval is performed using a FAISS index over Wikipedia with Sentence-BERT embeddings. We evaluate on 2WikiMultihopQA (strictly split 60% train, 20% val, 20% test to prevent leakage) for in-domain hop-level evaluation, and HotpotQA for out-of-domain generalization.

Metrics. We compare our system and the baseline on the held-out test sets, reporting: (1) *Failure detection* (Precision, Recall, $F_{0.5}$), (2) *First-failure localization accuracy*, (3) *Detection latency and token cost*, and (4) *End-to-end final-answer accuracy*.

6 Project Timeline

We propose a 9-week schedule:

Timeline	Milestones
Wks 1–2 (By Sep 10)	Data curation, hop labeling, counterfactuals.
Wks 3–4 (By Sep 24)	Probes (Ph 1) & Causal validation (Ph 2).
Wks 5–6 (By Oct 8)	Naive Restart (Ph 3) & Baseline (Ph 4).
Wks 7–8 (By Oct 22)	Full system evaluation (Ph 5).
Wk 9 (By Oct 31)	Fallback analysis and final paper write-up.

7 Conclusion

We propose to test whether an LLM’s internal hidden states can serve as a faster, more mechanistically faithful trigger for retrieval-augmented repair than existing text-based diagnosis. Our plan combines hop-level probing, causal verification via activation patching, and a real-time “Naive Restart” intervention, evaluated against a controlled, re-implemented text-based baseline. We view both a clear efficiency win and a more nuanced, mixed accuracy/efficiency result as valid, reportable outcomes.

References

- Ingeol Baek, Hwan Chang, ByeongJeong Kim, Jimin Lee, and Hwanhee Lee. 2025. Probing-RAG: Self-probing to guide language models in selective document retrieval. *Findings of the Association for Computational Linguistics: NAACL 2025*.
- Abhimanyu Dubey, Abhinav Jauhri, Abhinav Pandey, et al. 2024. The Llama 3 herd of models. *arXiv preprint arXiv:2407.21783*.
- Xanh Ho, Anh-Khoa Duong Nguyen, Saku Sugawara, and Akiko Aizawa. 2020. Constructing a multi-hop QA dataset for comprehensive evaluation of reasoning steps. In *Proceedings of the 28th International Conference on Computational Linguistics*.
- Shuguang Jiao, Chengkai Huang, Shuhan Qi, Xuan Wang, Yifan Li, and Lina Yao. 2026. Doctor-RAG: Failure-aware repair for agentic retrieval-augmented generation. *arXiv preprint arXiv:2604.00865*. ArXiv:2604.00865.
- Jeff Johnson, Matthijs Douze, and Hervé Jégou. 2019. Billion-scale similarity search with GPUs. *IEEE Transactions on Big Data*.
- Patrick Lewis, Ethan Perez, Aleksandra Piktus, Fabio Petroni, Vladimir Karpukhin, Naman Goyal, Heinrich Küttler, Mike Lewis, Wen-tau Yih, Tim Rocktäschel, Sebastian Riedel, and Douwe Kiela. 2020. Retrieval-augmented generation for knowledge-intensive NLP tasks. In *Advances in Neural Information Processing Systems*, volume 33.
- Hunter Lightman, Vineet Kosaraju, Yura Burda, Harri Edwards, Bowen Baker, Teddy Lee, Jan Leike, John Schulman, Ilya Sutskever, and Karl Cobbe. 2024. Let’s verify step by step. In *The Twelfth International Conference on Learning Representations*.
- Kevin Meng, David Bau, Alex Andonian, and Yonatan Belinkov. 2022. Locating and editing factual associations in GPT. In *Advances in Neural Information Processing Systems*, volume 35.
- Nils Reimers and Iryna Gurevych. 2019. Sentence-BERT: Sentence embeddings using Siamese BERT-networks. In *Proceedings of the 2019 Conference on Empirical Methods in Natural Language Processing*.
- Jesse Vig, Sebastian Gehrmann, Yonatan Belinkov, Sharon Qian, Daniel Nevo, Yaron Singer, and Stuart Shieber. 2020. Investigating gender bias in language models using causal mediation analysis. *Advances in Neural Information Processing Systems*, 33.
- An Yang, Baosong Yang, Beichen Zhang, et al. 2024. Qwen2.5 technical report. *arXiv preprint arXiv:2412.15115*.
- Zhilin Yang, Peng Qi, Saizheng Zhang, Yoshua Bengio, William W. Cohen, Ruslan Salakhutdinov, and Christopher D. Manning. 2018. HotpotQA: A dataset for diverse, explainable multi-hop question answering. In *Proceedings of the 2018 Conference on Empirical Methods in Natural Language Processing*.